

```
from google.colab import files
uploaded = files.upload()
```



Choose files 4th question.csv

- **4th question.csv**(text/csv) - 58052 bytes, last modified: 29/04/2025 - 100% done
Saving 4th question.csv to 4th question (1).csv

```
import pandas as pd
```

```
# Load the CSV file (change the filename if different)
df = pd.read_csv("4th question.csv")
```

```
# Preview the first few rows
print("\n🔍 First 5 rows:")
print(df.head())
```

```
# Show column names and data types
print("\n📄 Column Info:")
print(df.info())
```

```
# Check for missing values
print("\n! Missing Values:")
print(df.isnull().sum())
```



🔍 First 5 rows:

	Feed rate (mm/rev)	Spindle speed (RPM)	Tool material	\
0	0.10	1000	Carbide	
1	0.15	1200	Carbide	
2	0.20	1400	HSS	
3	0.25	1600	HSS	
4	0.30	1800	Carbide	

	Coolant usage (L/min)	Workpiece hardness (HB)	Cut time (min)	\
0	1.0	180	2	
1	1.5	190	3	
2	2.0	200	4	
3	2.5	210	5	
4	3.0	220	6	

	Tool path length (mm)	Tool usage frequency (uses/day)	Depth of cut (mm)	\
0	200	5	0.5	
1	250	6	0.6	
2	300	7	0.7	
3	350	8	0.8	
4	400	9	0.9	

	Vibration level (mm/s)	Tool wear (mm)
0	0.30	0.010
1	0.35	0.015
2	0.40	0.020
3	0.45	0.025
4	0.50	0.030

📄 Column Info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Feed rate (mm/rev)	1000 non-null	float64
1	Spindle speed (RPM)	1000 non-null	int64
2	Tool material	1000 non-null	object
3	Coolant usage (L/min)	1000 non-null	float64
4	Workpiece hardness (HB)	1000 non-null	int64
5	Cut time (min)	1000 non-null	int64
6	Tool path length (mm)	1000 non-null	int64
7	Tool usage frequency (uses/day)	1000 non-null	int64
8	Depth of cut (mm)	1000 non-null	float64
9	Vibration level (mm/s)	1000 non-null	float64
10	Tool wear (mm)	1000 non-null	float64

```
dtypes: float64(5), int64(5), object(1)
```

```
memory usage: 86.1+ KB
```

```
None
```

! Missing Values:

Feed rate (mm/rev)	0
Spindle speed (RPM)	0
Tool material	0
Coolant usage (L/min)	0
Workpiece hardness (HB)	0
Cut time (min)	0
Tool path length (mm)	0

```
# Strip extra spaces from column names
df.columns = [col.strip() for col in df.columns]

# Display cleaned column names
print("✅ Cleaned Columns:")
print(df.columns)

# Encode 'Tool material' if it exists and is categorical
if 'Tool material' in df.columns:
    df['Tool material'] = df['Tool material'].astype('category').cat.codes
    print("⚠️ Encoded 'Tool material' as numeric.")

# Drop rows with missing values (or use df.fillna() if you prefer)
df.dropna(inplace=True)

# Confirm data shape after cleaning
print(f"\n✅ Data shape after cleaning: {df.shape}")
```

```
✅ Cleaned Columns:
Index(['Feed rate (mm/rev)', 'Spindle speed (RPM)', 'Tool material',
      'Coolant usage (L/min)', 'Workpiece hardness (HB)', 'Cut time (min)',
      'Tool path length (mm)', 'Tool usage frequency (uses/day)',
      'Depth of cut (mm)', 'Vibration level (mm/s)', 'Tool wear (mm)'],
      dtype='object')
⚠️ Encoded 'Tool material' as numeric.

✅ Data shape after cleaning: (1000, 11)
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score

# Split features and target
X = df.drop(columns=["Tool wear (mm)"])
y = df["Tool wear (mm)"]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define models
models = {
    "Linear": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.1),
    "ElasticNet": ElasticNet(alpha=0.1, l1_ratio=0.5)
}

# Train and evaluate
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\n🧠 {name} Regression")
    print("R² Score:", r2_score(y_test, y_pred))
    print("MSE:", mean_squared_error(y_test, y_pred))
```

```
🧠 Linear Regression
R² Score: 1.0
MSE: 8.78664129632872e-31

🧠 Ridge Regression
R² Score: 1.0
MSE: 2.520470981418987e-25

🧠 Lasso Regression
R² Score: 0.99999999999985711
MSE: 2.9196079761707073e-12

🧠 ElasticNet Regression
R² Score: 0.9999999999996427
MSE: 7.299384894708484e-13
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set up the plot
plt.figure(figsize=(12, 10))
plt.suptitle("Actual vs Predicted Tool Wear", fontsize=16)

# Loop through models and plot predictions
```

```

for i, (name, model) in enumerate(models.items(), 1):
    y_pred = model.predict(X_test)

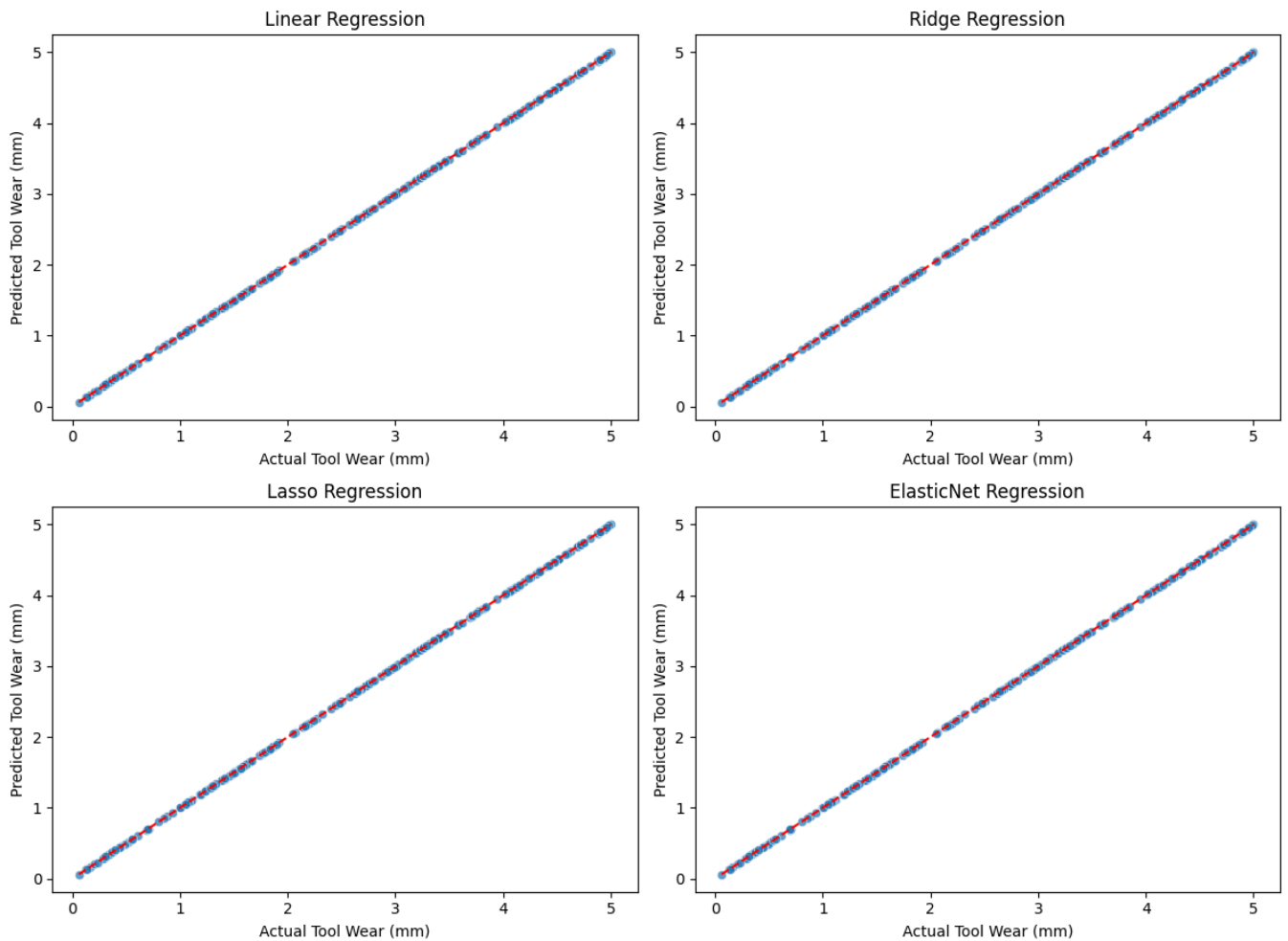
    plt.subplot(2, 2, i)
    sns.scatterplot(x=y_test, y=y_pred, alpha=0.7)
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    plt.xlabel("Actual Tool Wear (mm)")
    plt.ylabel("Predicted Tool Wear (mm)")
    plt.title(f"{name} Regression")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```



Actual vs Predicted Tool Wear



```

import pandas as pd

# Choose the best model (replace with your preferred one)
best_model = models["Lasso"] # You can change to "Linear", "Ridge", or "ElasticNet"

# Get feature names and importance
importance = best_model.coef_
features = X.columns

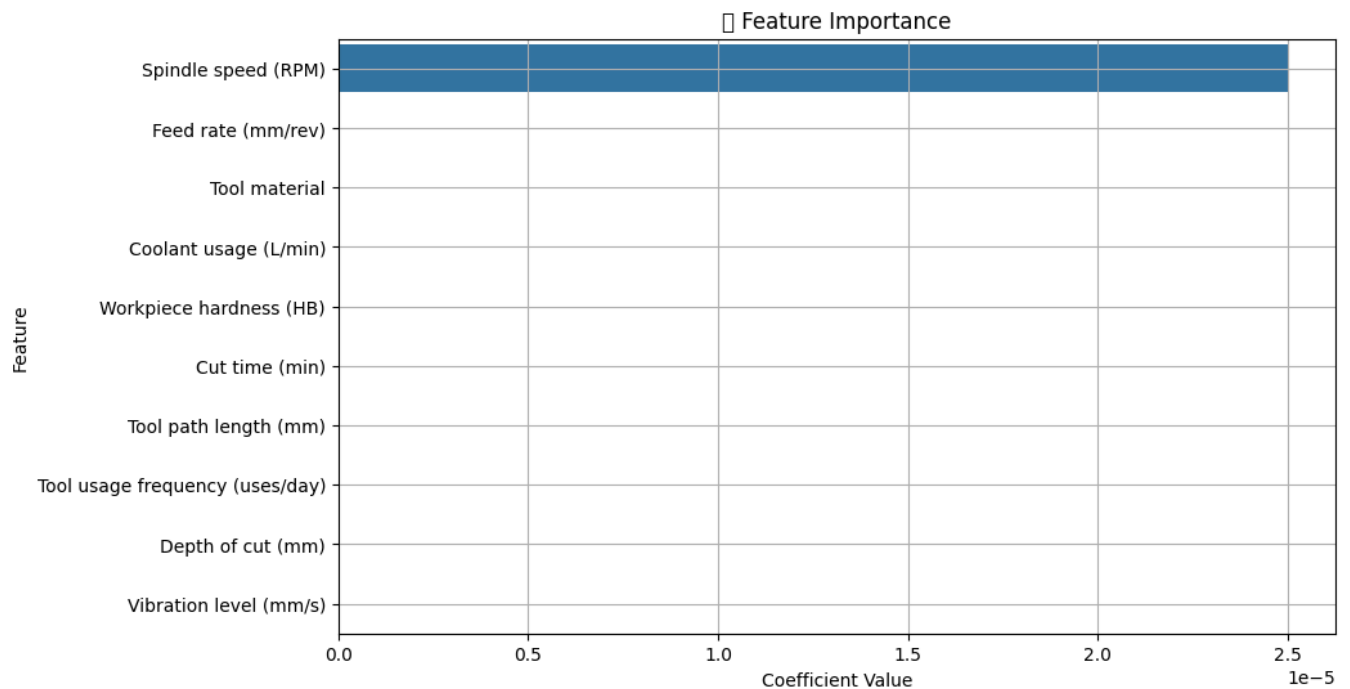
# Create a DataFrame for plotting
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importance
}).sort_values(by='Importance', key=abs, ascending=False)

# Plot

```

```
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title("🔍 Feature Importance")
plt.xlabel("Coefficient Value")
plt.ylabel("Feature")
plt.grid(True)
plt.show()
```

➞ /usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128269 (\N{LEFT-POINTING MAGNIFYING GLASS}) not found. Falling back to U+2600 (⬅).



```
import joblib

# Save the best model (here we use Lasso)
best_model = models["Lasso"] # Replace with your preferred model if necessary
joblib.dump(best_model, "tool_wear_model.joblib")

print("✅ Model saved successfully!")
```

➞ ✅ Model saved successfully!

```
# Load the model
loaded_model = joblib.load("tool_wear_model.joblib")

# Use the loaded model to make predictions
y_pred_loaded = loaded_model.predict(X_test)

# Evaluate the predictions
print("🔍 Loaded Model R² Score:", r2_score(y_test, y_pred_loaded))
```

➞ 🔍 Loaded Model R² Score: 0.9999999999985711

